

Lecture 3 Companion

What the Automata Are For

Lexer generation for readers who already know NFAs, DFAs and subset construction

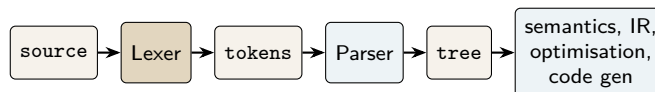
Compiler Construction Module

A short, compiler-focused reading of Lecture 3

Who this is for

You have taken an automata course. You can build Thompson's NFA, run subset construction, and minimise a DFA in your sleep. What you want to know is: *why is a compilers course making me do it again, where does it sit in the compiler, and what is actually new?* This note answers exactly that and nothing else. The full chapter (*Chapter 3: Automating Lexical Analysis*) has the proofs and the automata revision; this note has the plot.

1 Where we are in the compiler



Lectures 2 and 3

We are still entirely inside the **lexer**. Lecture 2 defined the job (characters in, tokens out) and gave the language for describing it (one regular expression per token class). Lecture 3 answers one question:

Given that list of regular expressions, how does a program come into existence that cuts the input into tokens?

The answer is “a **lexer generator** compiles the regular expressions into a DFA, and the lexer is a loop that runs that DFA.” The NFA, subset construction and minimisation are the internals of the generator. You never write them in a compiler; you write the spec and the tool runs them. That is why the lecture revisits them: not to teach automata, but to show that the tool is mechanical and trustworthy.

Key insight. Lecture 3 does not move past tokens. Its output is the same token stream Lecture 2 described. What changes is *who writes the lexer*: a tool does, from your regular expressions.

2 The generator in four lines

Mechanism: What flex / JFlex do with your spec

1. Take every token rule $R_1, R_2, \dots, R_n$ from the spec and form one regular expression $R_1 \mid R_2 \mid \dots \mid R_n$.
2. Regex $\rightarrow$ NFA (Thompson), NFA $\rightarrow$ DFA (subset construction), DFA $\rightarrow$ minimal DFA. *You know all three.*
3. Store the DFA as a table $\delta[\text{state}][\text{character}]$ plus, for each accepting state, *which token rule* it belongs to.
4. Emit a fixed driver loop (Section 4.3) that reads characters, follows the table, and returns one token at a time.

What the table actually looks like

Here is the complete table for the small spec used in Section 4:

```
"if" → IF    "==" → EQ    "=" → ASSIGN    "(" → LP    ")" → RP
[0-9]+ → INT  [a-z][a-z0-9]* → ID  [\t]+ → skip
```

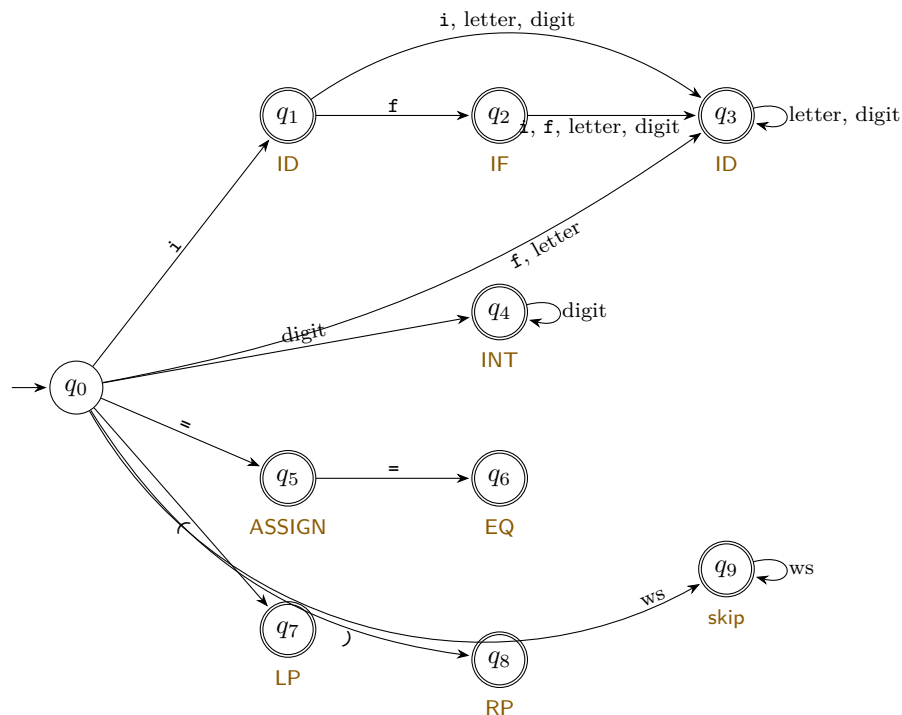
The generator groups characters into *classes* that the DFA never distinguishes (every digit behaves the same, every letter other than **i** and **f** behaves the same), so the table has one column per class rather than one per ASCII code. A blank cell is the error state $\emptyset$.

$\delta[q][c]$	i	f	letter	digit	=	(	)	ws	other	tag
q_0	q_1	q_3	q_3	q_4	q_5	q_7	q_8	q_9		–
q_1	q_3	q_2	q_3	q_3						ID
q_2	q_3	q_3	q_3	q_3						IF , ID
q_3	q_3	q_3	q_3	q_3						ID
q_4				q_4						INT
q_5					q_6					ASSIGN
q_6										EQ
q_7										LP
q_8										RP
q_9								q_9		skip

q_0 start q_5 read =
 q_1 read i q_6 read ==
 q_2 read if q_7 read (
 q_3 inside an identifier q_8 read)
 q_4 inside a number q_9 inside whitespace

Columns: *letter* means any lowercase letter other than **i** and **f**; *ws* is space or tab; *other* is any other character.

The same machine as a picture. Double circles are accepting states; the word under each is the token that state returns. Missing arrows go to the error state (a blank cell in the table).



Follow `if(` on the picture: start at q_0 , take the `i` arrow to q_1 , the `f` arrow to q_2 , then look for a `(` arrow out of q_2 : there is none, so the machine is stuck. The last double circle visited was q_2 , labelled **IF**. That is the token. Now follow `ifx`: same path to q_2 , then the “letter” arrow to q_3 (**ID**), then stuck on whatever comes next: the token is `ID("ifx")`, and the keyword state was simply passed through.

Three things to read off it.

- **The tag column is the whole difference from an automata-course DFA.** Rows with a tag are accepting; the tag says which token the driver returns. Row q_2 carries two tags because `if` is matched by both the keyword rule and the identifier rule; **IF** is bold because it comes first in the spec, so that is what q_2 returns.
- **Longest match is visible as “keep going while the row has a transition.”** From q_2 on any letter or digit the machine moves to q_3 : `iffy` is an identifier, not `if` followed by `fy`. From q_5 on `=` it moves to q_6 : `==` is one token. The driver only stops when it hits a blank cell.
- **Character classes keep the table small.** Ten rows by nine columns instead of ten rows by 128 columns; the lexer first maps a character to its class with a tiny lookup array, then indexes this table.

Worked example: Reading the table the way the driver does

Input `if(`: $q_0 \xrightarrow{i} q_1 \xrightarrow{f} q_2 \xrightarrow{(\text{}} \emptyset$. Stuck after two characters; the last accepting state was q_2 , tag **IF**. Return **IF**, leave `(` in the input.

Input `ifx=`: $q_0 \xrightarrow{i} q_1 \xrightarrow{f} q_2 \xrightarrow{x} q_3 \xrightarrow{=} \emptyset$. Last accepting state q_3 , tag **ID**. Return `ID("ifx")`. The keyword row was passed through without

stopping, which is exactly maximal munch.

Input ==7: $q_0 \xrightarrow{=} q_5 \xrightarrow{=} q_6 \xrightarrow{7} \emptyset$. Last accepting q_6 , tag EQ. Return EQ; the 7 will start the next call in q_0 and go to q_4 .

Input #: $q_0 \xrightarrow{\#} \emptyset$ with nothing on the stack and q_0 not accepting: lexical error, no rule matches #.

Only two ideas in this list are absent from an automata course, and they are the whole content of the lecture as far as a compiler writer is concerned:

- the DFA must report **which** rule matched, not just *that* something matched (Section 3);
- the DFA must find the **longest** token starting at the current position, not merely decide whether the whole input is in the language (Section 4).

Everything else is revision.

3 New idea 1: the DFA must say which token

In automata theory an accept state means “yes.” In a lexer, “yes” is useless: the parser needs to know whether the thing it just got is an IF, an ID or an INT.

Mechanism: Tagged accept states

When the branches $R_1 \mid \dots \mid R_n$ are combined into one NFA, their accept states are *not* merged. Each keeps a tag: the token type (and action) of its rule. Subset construction carries the tags along: a DFA state is accepting if it contains any tagged NFA accept state, and it inherits those tags.

The interesting case is when a DFA state inherits *two* tags. Take the two rules

"if" $\rightarrow$ IF and [a-z][a-z0-9]* $\rightarrow$ ID

and read the input `if`. The keyword branch reaches its accept state; the identifier branch *also* reaches its accept state, because `if` is a perfectly good identifier. The DFA state after `if` therefore contains both IF and ID. A lexer returns one token, so there must be a tie-break.

Concept: Priority = order in the spec

When a DFA state carries several tags, the rule written *earlier* in the specification wins. This is why every real spec lists keywords before the identifier rule. Write them after it and every keyword lexes as an identifier.

Worked example: Pause and check

Spec order: (1) $[a-z]^+ \rightarrow$ ID, (2) `"while"` $\rightarrow$ WHILE. Input `while`. Token? *Answer:* ID(`"while"`). Both rules match five characters; the tie goes to rule 1. The keyword is silently lost and the parser will later complain about a variable named `while` followed by a parenthesis. Swap the two

rules to fix it.

4 New idea 2: longest match, and how a DFA finds it

4.1 Why “accept as soon as you can” is wrong

An automaton decides whether an entire string belongs to a language. A lexer faces a stream and must decide *where the first token ends*. Consider the identifier rule on the input `infinity = 3`. After `i` the DFA is in an accepting state (`i` is an identifier). After `in`, still accepting. After `inf`, `infi`, ..., `infinity`, all accepting. After the space, no transition. The token is `infinity`, the *last* accepting position, not the first. Chapter 2 called this rule *maximal munch*; Lecture 3 shows the mechanism that implements it.

4.2 Why “read until stuck” is also wrong

Stuck is not the same as “just past the last token.” Suppose the spec contains `abc` and `(abc)*d`, and the input is `abcabx`. Reading greedily: `abc` (accepting, rule 1), `abca`, `abcab` (not accepting: rule 2 is still hoping for a `d`), then `x`: stuck. The last accepting position was three characters back. The lexer must *go back* to it and give the characters `ab` back to the input.

4.3 The scan-and-backtrack loop

This is the driver loop every generated lexer runs to produce *one* token. It is called once per token, with *i* pointing at the first unconsumed character.

Mechanism: One token, by scan and backtrack

Algorithm 1: `nextToken()`

```
1 start  $\leftarrow i$ ; q  $\leftarrow q_0$ ; clear the stack;
2 Phase 1: read ahead as far as the DFA allows;
3 while true do
4   c  $\leftarrow \text{input}[i]$ ;
5   if c = EOF or  $\delta(q, c) = \emptyset$  then break;
6   if q is accepting then clear the stack;
7   push q; q  $\leftarrow \delta(q, c)$ ; i  $\leftarrow i + 1$ ;
8 Phase 2: back up to the last accepting state;
9 while q is not accepting do
10  if stack is empty then error: no token here;
11  q  $\leftarrow \text{pop}()$ ; i  $\leftarrow i - 1$  (the character is unread);
12 return token with type = tag of q, text = input[start..i - 1];
```

Read the two phases in words. *Phase 1* is the ordinary DFA loop, except that it does not stop at accepting states; it remembers them (by clearing the stack) and pushes on. *Phase 2* undoes the overshoot: pop a state, give one character back, until the state is accepting. The tag of that state is the token type; the position i is one past the end of the token; the next call starts there.

Worked example: Full trace: abcabx with rules abc (R1) and (abc)*d (R2)

read	state after	accepting?	stack after step	comment
a	q_a	no	$[q_0]$	
b	q_{ab}	no	$[q_0, q_a]$	
c	q_{abc}	yes, R1	$[q_0, q_a, q_{ab}]$	R2 would still need a d
a	q_{abca}	no	$[q_{abc}]$	leaving an accept: stack cleared, then pushed
b	q_{abcab}	no	$[q_{abc}, q_{abca}]$	
x	stuck			break, nothing pushed

Phase 2: q_{abcab} not accepting, pop q_{abca} , unread b; not accepting, pop q_{abc} , unread a; accepting. Return R1("abc"), i now points at the second a. Next call lexes abx: it will return abc again and then fail on x, which is a lexical error (no rule matches x).

Worked example: Full trace of a realistic line

Spec, in this order: "if"→IF, "=="→EQ, "="→ASSIGN, "("→LP, ")"→RP, [0-9]+→INT, [a-z][a-z0-9]*→ID, [\t]+→skip.
Input: if (x1==10) y=x1

Key insight. The stack is wiped every time the DFA leaves an accepting state. So it only ever holds the states visited since the most recent accept. Backtracking through it lands exactly on that accept: the longest match, for free.

call	what Phase 1 reads, and where it stops	returned
1	if; then space: stuck. State tagged {IF, ID}; IF listed first	IF
2	space; then (: stuck	(skipped, no token)
3	(; then x: stuck	LP
4	x, x1; then =: stuck. Last accept at x1	ID("x1")
5	= (accepting: ASSIGN), == (accepting: EQ); then 1: stuck. Last accept at ==	EQ
6	1, 10; then): stuck	INT(10)
7	); then space: stuck	RP
8	space; then y: stuck	(skipped)
9	y; then =: stuck	ID("y")
10	= (ASSIGN); then x: stuck. Only one accept	ASSIGN
11	x, x1; then EOF	ID("x1")

Token stream handed to the parser: IF LP ID EQ INT RP ID ASSIGN ID. Three things to notice. Call 1 is decided by *priority* (tie in length). Call 5 is decided by *longest match* (== beats =), with no priority involved. Calls 2 and 8 return nothing because the whitespace rule's action is "skip"; the driver simply calls itself again.

Concept: The two tie-breaks, in the right order

1. **Longest match first.** The token is the longest prefix accepted by the combined DFA. Rule order plays no part in this step.
 2. **Priority only on equal length.** If the final state carries several tags, the earliest rule wins.
- if versus infinity: on if both IF and ID match 2 characters, tie, IF wins by order. On infinity ID matches 8 characters and IF only 2, no tie, ID wins outright. Priority never overrides length.

5 The one performance problem, and its fix

Backtracking costs time. On the spec `abc | (abc)*d` and input `abcabc...abc` (n characters, no `d`), every call to `nextToken()` reads to the end of the input hoping for a `d` and then backs up to its own `abc`. That is $n/3$ tokens, each reading $O(n)$: $\Theta(n^2)$ for a file of n characters. A large source file would lex noticeably slowly.

Mechanism: Reps' fix: remember dead ends

If backtracking ever passes through a non-accepting state q at input position i , then *no future call* can benefit from reading ahead from (q, i) : the input is the same and the DFA is the same, so that read-ahead will fail

again. Record it in a table `hopeless[q][i]`, and in Phase 1 stop as soon as the current (q, i) is marked. Each (q, i) pair is then explored at most once over the whole file, so total work is $O(|Q| \cdot n)$: linear.

On the bad input above, only the first token pays the full read-ahead; every later `abc` finds the states past its own end already marked hopeless and stops at once. This is the only genuinely new algorithmic content in the lecture (Reps, TOPLAS 1998), and it is what makes “maximal munch with a DFA” safe to ship.

6 What you can skip, what you cannot

Trade-offs: Triage of Lecture 3 for someone who knows automata

- **Skip (revision):** DFA definition and reading loop; NFA definition; Thompson’s construction; subset construction; minimisation by table filling. Know only that the generator runs them and that subset construction can blow up exponentially, which is why it is a separate, offline step done once per spec.
- **Read (new):** tagged accept states and rule priority (Section 3); longest match via scan-and-backtrack (Section 4); the quadratic trap and Reps’ memo table.
- **Optional:** Brzozowski derivatives as an alternative that builds the minimal DFA directly, and NFA simulation as what regex libraries do when the pattern is not fixed in advance.

7 What comes next

The token stream is done. The next lectures go one level up:

- **Parsing.** The parser consumes tokens and checks and builds structure: expressions, statements, blocks. Its specification language is a *context-free grammar* instead of regular expressions, because nesting (balanced parentheses, nested blocks) is not regular. Its machine is a pushdown automaton instead of a finite one; you will meet it as LL and LR parsing tables.
- **The same pattern repeats.** Just as Lecture 3 turned a regex spec into a table-driven lexer, a parser generator (`yacc`, `bison`, `ANTLR`, `CUP`) turns a grammar spec into a table-driven parser. The automata knowledge you already have is again the engine underneath, and again the compiler-specific additions are about *which* rule matched (which production to reduce by) and how to resolve ambiguity (precedence and associativity, the parser’s analogue of priority).

Summary: The whole lecture in five lines

1. We are still in lexical analysis; the output is Lecture 2's token stream.
2. A lexer generator compiles $R_1 \mid \dots \mid R_n$ into a minimal DFA by the constructions you already know, done once, offline.
3. Accept states carry the token type of their rule; ties on equal length go to the rule written first.
4. The generated lexer reads ahead until stuck and backtracks to the last accepting state, which implements longest match; the stack cleared at each accept is the trick.
5. Memoising hopeless (q, i) pairs keeps that loop linear. Next: parsing, the same story with grammars and pushdown automata.